SIMATS ENGINEERING

ASSESSMENT TOOL 1: Constraint-Based Problem Solving

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1506

COURSE OUTCOME COVERED

CO5: *Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN.* (BL5)

Assessment Tool Weightage: 40%

Dave's Taxonomy: Articulation (Level 4) **Total Marks: 50**
Question Count: 5

Code of Conduct

I K Sumanth Kumar Reddy (192425222) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: K Sumanth

Assessment Tasks

Constraint-Based Problem Solving

1. Design an HDFS-based storage solution for a media platform under the constraints that data availability must remain above 99.9%, replication factor cannot exceed 3, and storage growth must stay cost efficient. Justify your design.
2. A MapReduce workflow must process log data within a 20-minute batch window using limited cluster nodes. Propose a job design under these constraints and explain task distribution.
3. A YARN-managed cluster supports ETL, reporting, and machine learning jobs. Design a scheduler policy under the constraints of fairness, priority for ETL, and recovery from node failure.
4. An enterprise needs to ingest data from SAN, NAS, and operational databases into a big data platform while maintaining backup reliability and minimal duplication. Propose a constrained architecture and justify each component.
5. Design an end-to-end Hadoop ecosystem solution under the constraints of secure data movement, high availability, and integration with Apache NiFi or ETL pipelines. Explain how your design satisfies all constraints.

Constraint-Based Problem Solving Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Constraints	25%	All technical constraints are identified and prioritized accurately	Most constraints identified	Some constraints identified	Constraints poorly understood
System Design Quality	25%	Design is robust, feasible, and aligned to constraints	Reasonable design	Basic design with gaps	Weak design
Application of Hadoop Concepts	20%	Uses HDFS, MapReduce, YARN, and integration concepts effectively	Mostly effective use	Partial use of concepts	Weak concept application
Justification	15%	Strong technical reasoning for all choices	Good justification	Basic justification	Little justification
Clarity	15%	Clear, well-structured, and professional answer	Mostly clear	Somewhat unclear	Poorly structured

ANSWERS

1. HDFS Media Platform
Constraints:
- Availability > 99.9%
- Maximum replication factor = 3
- Cost-efficient storage

Design:
- Use HDFS with replication factor **3**.
- Store replicas across different DataNodes and racks.
- Enable NameNode High Availability.
- Use commodity servers to control costs.
- Monitor under-replicated blocks.
- Use compression for suitable media metadata.
- Use appropriate storage tiers for frequently and rarely accessed data.

Justification:
Replication factor 3 provides fault tolerance while staying within the specified limit. Rack-aware placement protects against rack failure. NameNode HA avoids a single metadata failure point.

2. MapReduce Within 20 Minutes
Constraint: Process logs within a 20-minute batch window using limited nodes.
Design:
- Divide logs into balanced input splits.
- Use multiple mapper tasks to process splits in parallel.
- Use a combiner to reduce intermediate data.
- Select an appropriate number of reducers.
- Partition data evenly among reducers.
- Compress intermediate data to reduce network traffic.
- Avoid unnecessary data transfers.
- Monitor task execution and resource utilization.

Workflow:
HDFS Logs → Input Splits → Mappers → Combiner → Shuffle/Sort → Reducers → Output
This design maximizes parallel processing despite the limited number of nodes.

3. YARN Scheduler Policy
The cluster supports **ETL, reporting and machine learning**.
Policy:

- Give **ETL jobs high priority**.
- Allocate dedicated resources/queues for different workloads.
- Ensure reporting receives guaranteed minimum resources.
- Allow ML jobs to use remaining resources.
- Apply fair sharing within each queue.
- Set maximum resource limits.
- Enable application retry and recovery.
- Use node failure detection and task reallocation.

Example priority:
ETL → Reporting → Machine Learning
However, resource minimums should prevent lower-priority workloads from being completely starved.

4. SAN, NAS and Operational Database Ingestion
Architecture:
SAN + NAS + Operational DB → Apache NiFi/ETL → Validation/Deduplication → HDFS/Data Lake → Backup → Analytics

- Use **Apache NiFi** to ingest files and database records.
- Use ETL for transformation and validation.
- Assign unique identifiers to records.
- Perform deduplication before storage.
- Store large datasets in HDFS.
- Maintain metadata about source and version.
- Use replication for backup reliability.
- Maintain separate backup copies.
- Monitor data transfer and failures.

Result: The architecture supports reliable ingestion while reducing duplicate records.

5. End-to-End Hadoop Ecosystem
Architecture:
Data Sources → Secure NiFi/ETL → HDFS → YARN → MapReduce/Processing → Analytics

Security

- Encrypt data during transmission.
- Authenticate users and services.
- Apply authorization and ACLs.
- Encrypt sensitive stored data.
- Maintain audit logs.

High Availability

- HDFS replication.
- NameNode High Availability.
- ResourceManager recovery.
- Multiple DataNodes.

NiFi/ETL Integration

- NiFi collects data from multiple sources.
- ETL cleans and transforms data.
- Processed data is stored in HDFS.
- YARN manages processing resources.
- MapReduce performs distributed processing.

Result: The design provides secure data movement, fault tolerance, scalable storage and reliable big data processing.